Module 3: Deutsch's Algorithm

Quantum Computing Workshop

March 20, 2025

Introduction to Quantum Algorithms

- Classical algorithms: Sequence of instructions for classical computers
- Quantum algorithms: Leverage quantum mechanics for computational advantage
- Potential advantages:
    - Superposition: Process multiple inputs simultaneously
    - Interference: Enhance correct answers, suppress wrong ones
    - Entanglement: Correlate results across computation
- Quantum algorithms often provide speedup for specific problems

Deutsch's Problem

- First quantum algorithm with provable speedup (1985)
- Problem statement:
  - Given a black-box (oracle) function $f : \{0,1\} \to \{0,1\}$
  - Determine if f is constant or balanced
  - Constant: $f(0) = f(1)$
  - Balanced: $f(0) \neq f(1)$
- Classically: Need to evaluate f twice (on both inputs)
- Quantum: Only one evaluation of f needed

Possible Functions

The four possible binary functions $f : \{0, 1\} \to \{0, 1\}$:

x	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	1	1
1	0	1	0	1
Type	Constant	Balanced	Balanced	Constant

- f_1 and f_4 are constant functions
- f_2 and f_3 are balanced functions
- Can we determine the function type with just one query?

Classical Solution

- Classical approach requires 2 function evaluations
  1. Compute $f(0)$
  2. Compute $f(1)$
  3. Compare results: if equal, f is constant; if different, f is balanced
- No way to reduce evaluations classically
- Need to evaluate both inputs in worst case
- This provides our benchmark for quantum speedup

Quantum Oracles

- Quantum algorithms interact with functions through "oracles"
- Oracles must be implemented as unitary operations
- Standard implementation: $U_f \left|x\right\rangle \left|y\right\rangle = \left|x\right\rangle \left|y \oplus f(x)\right\rangle$ where $\oplus$ is addition modulo 2 (XOR)
- This operation is reversible (unlike classical function evaluation)
- For Deutsch's algorithm, we use a simplified oracle with one input and one output qubit

Phase Kickback Trick

- When the second qubit is in state $|-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$:

$$
\begin{aligned}
U_f |x\rangle |-\rangle &= |x\rangle \frac{|0 \oplus f(x)\rangle - |1 \oplus f(x)\rangle}{\sqrt{2}} && (1) \\
&= |x\rangle \frac{|f(x)\rangle - |1 \oplus f(x)\rangle}{\sqrt{2}} && (2) \\
&= (-1)^{f(x)} |x\rangle |-\rangle && (3)
\end{aligned}
$$

- The function value $f(x)$ appears as a phase factor
- This converts the function evaluation into a phase shift
- Crucial technique for many quantum algorithms

Deutsch's Algorithm: Steps

1. Initialize two qubits: $|0\rangle\,|1\rangle$
2. Apply Hadamard gates to both qubits: $|+\rangle\,|-\rangle$
3. Apply the oracle: $\sum_{x\in\{0,1\}} \frac{(-1)^{f(x)}|x\rangle}{\sqrt{2}}\,|-\rangle$
4. Apply Hadamard to the first qubit: $|\psi_{final}\rangle$
5. Measure the first qubit

- If measurement result is $|0\rangle$, function is constant
- If measurement result is $|1\rangle$, function is balanced

Mathematical Analysis

After the oracle application, the first qubit is in state:

$$\frac{(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle}{\sqrt{2}} \tag{4}$$

Applying Hadamard to this state:

$$H\left(\frac{(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle}{\sqrt{2}}\right) \tag{5}$$

$$= \frac{1}{2}\left[(-1)^{f(0)}(|0\rangle + |1\rangle) + (-1)^{f(1)}(|0\rangle - |1\rangle)\right] \tag{6}$$

$$= \frac{1}{2}\left[(|0\rangle\left((-1)^{f(0)} + (-1)^{f(1)}\right) + |1\rangle\left((-1)^{f(0)} - (-1)^{f(1)}\right)\right] \tag{7}$$

Results Interpretation

Final state analysis:

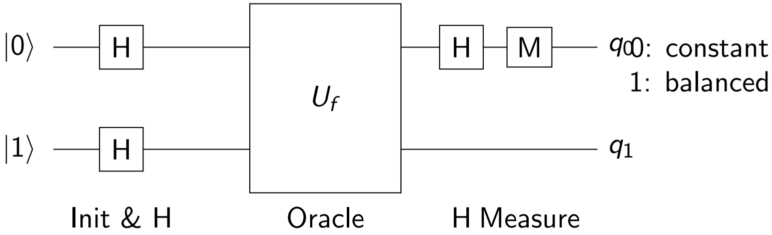
- If f is constant: $f(0) = f(1)$

$$\Rightarrow (-1)^{f(0)} = (-1)^{f(1)} \tag{8}$$
$$\Rightarrow |\psi_{final}\rangle = \pm |0\rangle \tag{9}$$

- If f is balanced: $f(0) \neq f(1)$

$$\Rightarrow (-1)^{f(0)} = -(-1)^{f(1)} \tag{10}$$
$$\Rightarrow |\psi_{final}\rangle = \pm |1\rangle \tag{11}$$

Measuring the first qubit determines if f is constant or balanced with 100% probability!


$|0\rangle$
H
U_f
H
M
q_0: constant
1: balanced
$|1\rangle$
H
q_1
Init & H
Oracle
H Measure

Implementation of Quantum Oracles

The four possible oracle implementations for Deutsch's problem:

Function	**Type**	**Circuit Implementation**
$f_1(x) = 0$	Constant	Identity gate (do nothing)
$f_2(x) = x$	Balanced	CNOT gate
$f_3(x) = \neg x$	Balanced	CNOT + X gate on target
$f_4(x) = 1$	Constant	X gate on target

- Each oracle can be efficiently implemented using simple quantum gates
- The oracle structure remains hidden during algorithm execution

Quantum Parallelism

- Key insight: How does Deutsch's algorithm evaluate f at both inputs with one query?
- Answer: Quantum superposition enables parallel function evaluation
- When we apply U_f to $|+\rangle|-\rangle$, we are simultaneously evaluating $f(0)$ and $f(1)$
- Mathematically:

$$U_f(|+\rangle|-\rangle) = U_f\left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}|-\rangle\right) \tag{12}$$

$$= \frac{1}{\sqrt{2}}[U_f(|0\rangle|-\rangle) + U_f(|1\rangle|-\rangle)] \tag{13}$$

$$= \frac{1}{\sqrt{2}}[(-1)^{f(0)}|0\rangle|-\rangle + (-1)^{f(1)}|1\rangle|-\rangle] \tag{14}$$

Limitations and Extensions

- Deutsch's algorithm provides only a modest speedup (2x)
- Limited to a specific type of problem
- Extensions:
  - Deutsch-Jozsa algorithm: n-bit version with exponential speedup
  - Determines if $f : \{0,1\}^n \to \{0,1\}$ is constant or balanced
  - Classical: up to $2^{n-1} + 1$ evaluations in worst case
  - Quantum: Just 1 evaluation
- Historical significance:
  - First demonstration of quantum advantage
  - Introduced key techniques used in many later algorithms
  - Inspired development of more powerful quantum algorithms

Key Takeaways

- Deutsch's algorithm solves a specific problem with one function evaluation instead of two
- Key techniques:
  - Quantum parallelism through superposition
  - Phase kickback to encode function values
  - Interference to extract global property
- Important historical milestone in quantum computing
- Demonstrates fundamental quantum advantage
- Building block for more powerful algorithms

Next Steps

- Implementing Deutsch's algorithm in Qiskit
- Introduction to Shor's algorithm